

RAG Production Stack — Comprehensive Technical Overview

Production-Oriented Retrieval-Augmented Generation Infrastructure

A client-readable and engineer-ready reference for authenticated RAG services, network isolation, observability, security and healthcare scanning, resource governance, backup recovery, verification boundaries, and developer handoff.

LightRAG | Zero-Trust Access | Metrics, Logs, and Traces | Profile-Based Scanning

Source-Reconciled Public Technical Edition | July 2026

Developed by Zahidul Islam

Current Evidence and Scope Note — July 2026

This edition was reconciled against the current non-secret repository sources. It documents architecture and configuration intent without claiming that a particular live environment is deployed, healthy, source-aligned, certified, or processing production data.

Verified repository snapshot

- 23 Docker Compose services: 9 core and 14 profile-activated.
- Three declared networks: frontend, backend, and monitoring.
- Nine core services include source-defined health checks.
- Prometheus uses a 15-second scrape interval with five alert rules.
- Loki retention is configured for 744 hours, equivalent to 31 days.
- Restic scripts define encrypted backup and isolated restore-verification workflows.

Safety and verification boundary

No container was started, stopped, restarted, built, or reconfigured during this documentation pass. Secret-bearing environment files, authentication databases, certificate material, backup contents, and generated scanner reports were excluded. Security and compliance tooling provides assessment capability; its presence does not establish certification, legal compliance, or a successful current scan.

1. EXECUTIVE SUMMARY

The RAG Production Stack is a production-hardened Retrieval-Augmented Generation (RAG) infrastructure designed for enterprise deployment. Built on LightRAG v1.4.6 and RAG-Anything v1.2.10 with Google Gemini 2.5 Flash as the LLM backbone, this stack delivers a fully containerized, security-first platform that combines knowledge graph-powered RAG with multimodal document processing capabilities.

The system provides zero-trust authentication via Authelia with two-factor enforcement, a comprehensive observability pipeline spanning metrics, logs, traces, and alerts, and built-in compliance scanning for HIPAA, SOC 2, and GDPR frameworks. All services are orchestrated through Docker Compose and calibrated to run on an 8GB RAM host with approximately 4.3GB allocated to containers.

Key Metrics:

- 23 containerized services (9 core + 14 on-demand via profiles)
- 3 Docker networks with strict isolation (frontend bridge, backend internal, monitoring internal)
- 7 Prometheus scrape targets at 15-second intervals
- 5 alert rules (container down, high memory, high CPU, target missing, Loki errors)

- 3 Grafana datasources (Prometheus, Loki, Jaeger) auto-provisioned
- 31-day log retention via Loki
- Zero-trust 2FA on every authenticated route
- TLS 1.2/1.3 with HSTS, CSP, and 6 security headers
- Rate limiting: 10 req/s per IP with burst 20
- 6 compliance scanning tools (Checkov, Trivy, SonarQube, OWASP ZAP, HAPI FHIR, Inferno ONC G10)
- Restic encrypted backups (7 daily, 4 weekly, 6 monthly retention)
- 4 git commits (sanitized for public release)

2. ARCHITECTURE OVERVIEW

2.1 High-Level Architecture:

The stack follows a layered architecture with strict network isolation across three Docker networks:

Frontend Network (bridge): Exposes Nginx reverse proxy to the host on ports 80 and 443, handling all inbound traffic with TLS termination, rate limiting, and forward authentication via Authelia.

Backend Network (internal): Hosts the core RAG services including LightRAG and RAG-Anything. These services are never exposed directly to the host, only accessible through Nginx.

Monitoring Network (internal): Contains the full observability stack (Prometheus, Grafana, Loki, Jaeger, AlertManager) isolated from application traffic.

2.2 Request Flow:

1. Client sends HTTPS request to Nginx (ports 80/443)
2. Nginx performs TLS termination with TLS 1.2/1.3 strict ciphers
3. Forward-auth subrequest is sent to Authelia for identity verification
4. Authelia enforces two-factor authentication (TOTP/WebAuthn)
5. Authenticated requests are proxied to the appropriate backend service
6. Responses flow back through Nginx with security headers (HSTS, CSP, X-Frame-Options)

3. CORE COMPONENTS

3.1 Service Registry:

Core Services (9, always running):

LightRAG (ghcr.io/hkuds/lightrag:1.4.6): Knowledge graph RAG engine. Port 9621. Memory 1024MB, CPU 2.0

Nginx (nginx:1.28-alpine): Reverse proxy, TLS termination, rate limiting. Ports 80/443. Memory 128MB, CPU 0.5

Authelia (authelia/authelia:4.39.15): 2FA/TOTP/WebAuthn authentication. Port 9091. Memory 128MB, CPU 0.5

Prometheus (prom/prometheus:v2.53.5): Metrics collection (7 targets, 15s scrape). Port 9090. Memory 384MB, CPU 1.0

Grafana (grafana/grafana:10.2.3): Dashboards + auto-provisioned datasources. Port 3000. Memory 384MB, CPU 1.0

Loki (grafana/loki:3.6.0): Log aggregation (31-day retention, non-blocking driver). Port 3100. Memory 384MB, CPU 1.0

Jaeger (jaegertracing/all-in-one:1.53): Distributed tracing via OpenTelemetry. Port 16686. Memory 256MB, CPU 1.0

AlertManager (prom/alertmanager:v0.27.0): Alert routing and notification. Port 9093. Memory 128MB, CPU 0.5

SonarQube (sonarqube:10.8-community): Static code analysis. Port 9000. Memory 1536MB, CPU 2.0

On-Demand Services (14, profile-activated):

Profile "rag" (1 service):

RAG-Anything (custom Dockerfile, Python 3.11): Multimodal document processor. Memory 1024MB

Profile "scan" (12 services):

Checkov (bridgecrew/checkov:latest): IaC compliance (HIPAA/SOC 2/GDPR)

Trivy (aquasec/trivy:0.69.3): Container CVE vulnerability scanning

Prowler (prowlercloud/prowler:latest): AWS compliance scanning

Prowler-GCP (prowlercloud/prowler:latest): GCP compliance scanning

Prowler-Azure (prowlercloud/prowler:latest): Azure compliance scanning

OWASP ZAP (ghcr.io/zaproxy/zaproxy:stable): Dynamic application security testing. Memory 512MB

HAPI FHIR (hapiproject/hapi:latest): FHIR R4 validator. Memory 1024MB

Inferno (custom Dockerfile, Ruby 3.3.6): ONC Certification g(10) FHIR test kit. Memory 1536MB

Inferno Worker (custom Dockerfile, Ruby 3.3.6): Async test runner (Sidekiq). Memory 512MB

Inferno Redis (redis:7-alpine): Cache for Inferno. Memory 128MB

Inferno Validator (infernocommunity/inferno-resource-validator:1.0.67): FHIR resource validation. Memory 768MB

Healthcare Scanner (custom Dockerfile, Alpine 3.21): Custom 40+ healthcare compliance rules. Memory 256MB

Profile "certbot" (1 service):

Certbot (certbot/certbot:v3.3.0): SSL certificate auto-renewal

Total: 9 core + 14 on-demand = 23 services

3.2 LightRAG Engine:

LightRAG v1.4.6 serves as the core retrieval-augmented generation engine. It leverages knowledge graphs to provide context-aware responses that go beyond traditional vector similarity search. The engine is allocated 1024MB of memory and 2 CPU cores, exposed internally on port 9621 and accessible only through the authenticated Nginx proxy.

Key capabilities: knowledge graph construction from ingested documents, hybrid retrieval combining graph traversal with vector search, integration with Google Gemini 2.5 Flash for high-quality low-latency generation.

3.3 RAG-Anything (Multimodal Processor):

RAG-Anything v1.2.10 extends the stack with multimodal document processing, enabling ingestion of PDFs, Office documents, images, tables, and mathematical equations. It builds multimodal knowledge graphs that enhance the retrieval quality of LightRAG.

Usage modes: single-file processing, batch directory processing, and hybrid query mode (naive, local, global, hybrid).

4. SECURITY ARCHITECTURE

4.1 Zero-Trust Authentication:

Every request must be authenticated regardless of network origin. Authelia v4.39.15 provides:

- Two-factor enforcement via TOTP and WebAuthn on all routes
- Forward-auth integration with Nginx (every proxied request triggers auth subrequest)
- Session management: 1-hour expiration, 5-minute inactivity timeout
- Brute force protection: 3 retries, 120s detection window, 300s lockout
- User database with Argon2id password hashing (memory-hard KDF)

4.2 Transport Security:

- TLS 1.2 and TLS 1.3 with strict cipher configuration (HIGH:!aNULL:!MD5)
- SSL certificates managed by Certbot v3.3.0 with automated renewal
- HTTP Strict Transport Security (HSTS) enforced on all responses (max-age 1 year, includeSubDomains, preload)

4.3 Security Headers:

All Nginx responses include:

- HSTS with max-age enforcement and preload
- Content-Security-Policy (CSP) for XSS mitigation
- X-Frame-Options: DENY (prevents clickjacking)
- X-Content-Type-Options: nosniff (prevents MIME sniffing)
- Referrer-Policy: strict-origin-when-cross-origin
- Permissions-Policy: camera=(), microphone=(), geolocation=()

4.4 Network and Container Hardening:

- Rate Limiting: 10 requests/second per IP with burst allowance of 20
- Network Isolation: 3 Docker networks (frontend bridge, backend internal, monitoring internal)
- Container Hardening: Read-only root filesystems, no-new-privileges flag, capability dropping on all containers
- Secrets Management: All credentials externalized via environment variables, no secrets in version control

4.5 Vulnerability and Compliance Scanning:

On-demand scanning tools activated via Docker Compose profiles:

- Checkov: IaC compliance for HIPAA, SOC 2, GDPR
- Trivy v0.69.3: Container image CVE scanning
- SonarQube 10.8: Static code analysis and security hotspots
- OWASP ZAP: Dynamic application security testing
- HAPI FHIR: FHIR R4 validation
- Inferno: ONC Certification g(10) FHIR test kit
- Healthcare Scanner: Custom 40+ healthcare compliance rules
- Prowler: Multi-cloud compliance (AWS + GCP + Azure)

Reports generated into reports/ directory with subdirectories for HIPAA, SOC 2, GDPR.

5. OBSERVABILITY STACK

Full-stack observability across four pillars: metrics, logs, traces, and alerts.

5.1 Metrics (Prometheus v2.53.5):

7 scrape targets at 15-second intervals:

- Prometheus (localhost:9090): Self-monitoring
- LightRAG (lightrag:9621): RAG engine metrics
- Grafana (grafana:3000/metrics): Dashboard metrics
- Nginx (nginx:80/nginx_status): Reverse proxy status
- Jaeger (jaeger:14269/metrics): Tracing metrics
- SonarQube (sonarqube:9000/api/system/status): Code analysis metrics
- Loki (loki:3100/metrics): Log aggregation metrics

5.2 Logging (Loki 3.6.0):

- Aggregates logs from all containers via Loki Docker driver plugin
- Non-blocking mode (logging never impacts application performance)
- 31-day retention (744 hours)
- Queryable through Grafana using LogQL
- Ingestion rate: 4 MB/s, burst: 6 MB/s

5.3 Distributed Tracing (Jaeger 1.53):

- OpenTelemetry integration via gRPC (ports 4317/4318)
- Jaeger UI on port 16686
- End-to-end request tracing across services

5.4 Alerting (AlertManager v0.27.0):

5 pre-configured alert rules:

- ContainerDown: Fires after 1 minute of service being unreachable (CRITICAL)
- HighMemoryUsage: Fires when any container exceeds 400MB for 5 minutes (WARNING)
- HighCPUUsage: Fires when CPU utilization exceeds 80% for 5 minutes (WARNING)
- PrometheusTargetMissing: Fires when target unreachable for 5 minutes (CRITICAL)
- LokiRequestErrors: Fires when Loki 5xx error rate exceeds 0.1 for 5 minutes (WARNING)

Alert routing: group by alertname + severity, 30s group wait, 5m group interval, 4h repeat. Critical alerts suppress warnings with same name.

5.5 Dashboards (Grafana 10.2.3):

3 auto-provisioned datasources:

- Prometheus (default): Metrics
- Loki: Logs
- Jaeger: Traces

Pre-built Docker Services Overview dashboard with panels for: services up/down, targets status, memory usage, CPU usage.

6. DEPLOYMENT GUIDE

6.1 Prerequisites:

- Docker Desktop 4.0+ with at least 4.5GB memory and 4 CPUs recommended
- Loki Docker driver plugin: docker plugin install grafana/loki-docker-driver:latest --alias loki --grant-all-permissions
- SSL certificates (fullchain.pem, privkey.pem) for TLS termination
- Google Gemini API key for LLM-powered RAG operations

6.2 Configuration:

Primary configuration via configs/.env:

- GEMINI_API_KEY: Google Gemini API key for LightRAG and RAG-Anything
- GRAFANA_ADMIN_PASSWORD: Grafana admin access
- Scanning service paths: DEV_PATH, TERRAFORM_PROD_PATH, AWS_CONFIG_DIR, GCP_KEY_PATH
- Azure credentials (if applicable): CLIENT_ID, CLIENT_SECRET, TENANT_ID, SUBSCRIPTION_ID

6.3 Deployment Commands:

Start full stack: `cd configs && docker compose up -d`
 Run HIPAA scan: `docker compose --profile scan run --rm checkov`
 Run CVE scan: `docker compose --profile scan run --rm trivy`
 Renew SSL: `docker compose --profile certbot up certbot`
 Build RAG-Anything: `docker compose --profile rag build raganything`

6.4 Post-Deployment:

- Change default SonarQube admin password (default: admin/admin)
- Rotate Gemini API key if .env was ever committed
- Schedule backup.sh via Windows Task Scheduler (daily 2:00 AM)
- Monthly backup verification via verify-backup.sh

7. ACCESS AND ENDPOINTS

7.1 Protected Routes (Authelia 2FA Required):

LightRAG: <https://healthcodeanalysis.com/lightrag/>
 Grafana: <https://healthcodeanalysis.com/grafana/>
 Jaeger: <https://healthcodeanalysis.com/jaeger/>
 Prometheus: <https://healthcodeanalysis.com/prometheus/>
 Authelia: <https://healthcodeanalysis.com/authelia/> (login portal, no auth required)

7.2 Local-Only Services (not exposed externally):

SonarQube: <http://localhost:9000> (static code analysis admin)
 AlertManager: <http://localhost:9093> (alert routing dashboard)
 LightRAG Direct: <http://localhost:9621> (bypasses Nginx, development only)
 Loki API: <http://localhost:3100> (log queries)

Jaeger OTEL: <http://localhost:4317/4318> (trace ingest receivers)

8. RAG-ANYTHING USAGE

Multimodal document processing for PDFs, images, tables, and equations. Activated via Docker Compose rag profile.

Processing Commands:

Single file: `docker compose --profile rag run --rm raganything /app/data/inputs/yourfile.pdf`

Batch directory: `docker compose --profile rag run --rm raganything --dir /app/data/inputs/`

Query hybrid mode: `docker compose --profile rag run --rm raganything --query "What are the main findings?" --mode hybrid`

Supported formats: PDF, DOCX, XLSX, PPTX, images, tables, equations

Query modes: naive, local, global, hybrid (recommended)

9. RESOURCE ALLOCATION

Calibrated for 8GB RAM host with approximately 4.3GB allocated:

Core Services:

LightRAG: 1024MB memory, 2.0 CPU

SonarQube: 1536MB memory, 2.0 CPU

Prometheus: 384MB memory, 1.0 CPU

Grafana: 384MB memory, 1.0 CPU

Loki: 384MB memory, 1.0 CPU

Jaeger: 256MB memory, 1.0 CPU

Nginx: 128MB memory, 0.5 CPU

Authelia: 128MB memory, 0.5 CPU

AlertManager: 128MB memory, 0.5 CPU

All services include Docker health checks with configurable intervals and retries.

10. TECHNOLOGY STACK SUMMARY

Codebase composition: Shell scripts (77.2%), Python (20.0%), Dockerfiles (2.8%)

Core Technologies:

LLM Provider: Google Gemini 2.5 Flash

RAG Engine: LightRAG 1.4.6

Multimodal Processor: RAG-Anything 1.2.10

Reverse Proxy: Nginx 1.28-alpine

Authentication: Authelia 4.39.15 (2FA/TOTP/WebAuthn)

Container Orchestration: Docker Compose 4.0+

Metrics: Prometheus v2.53.5

Logging: Loki 3.6.0 + Docker Driver

Tracing: Jaeger 1.53 + OpenTelemetry

Dashboards: Grafana 10.2.3

Code Analysis: SonarQube 10.8-community

IaC Scanning: Checkov (latest)

CVE Scanning: Trivy 0.69.3

SSL Management: Certbot v3.3.0

Backup: Restic (encrypted, incremental)

11. MAINTENANCE AND OPERATIONS

11.1 Backup Strategy:

Automated via scripts/backup.sh using Restic (encrypted, deduplicated, incremental).

Scope: configs/, LightRAG data, Grafana dashboards and provisioning.

Retention: 7 daily, 4 weekly, 6 monthly snapshots. Auto-prune after each backup.

Schedule: Windows Task Scheduler, daily at 2:00 AM.

Verification: Monthly via scripts/verify-backup.sh.

11.2 SSL Certificate Renewal:

Certbot v3.3.0 handles renewal. Nginx serves ACME challenge files for automated domain validation.

Run periodically to keep certificates valid.

11.3 Log Retention:

Loki retains logs for 31 days (744 hours). Configurable in monitoring/loki/loki-config.yml. Older logs auto-purged.

11.4 Security Updates:

Regular container image updates via Trivy scans. SonarQube for code quality regression. Update service versions in Docker Compose as needed.

12. CONCLUSION

The RAG Production Stack delivers comprehensive, enterprise-grade infrastructure for deploying Retrieval-Augmented Generation at scale. By combining LightRAG and RAG-Anything with a hardened security layer, full observability pipeline, and compliance scanning tools, the stack is ready for production workloads in regulated industries including healthcare (HIPAA), finance (SOC 2), and data privacy (GDPR).

The modular Docker Compose architecture allows individual components to be scaled, replaced, or extended independently, making the stack adaptable to evolving requirements and technology upgrades.

RAG Production Stack

Enterprise-Grade RAG Infrastructure with Zero-Trust Security and Full Observability

Built with Docker | LightRAG | Authelia | Prometheus | Grafana | Jaeger | Checkov | Trivy

Developed by Zahidul Islam

zahidul-islam.vercel.app | github.com/ZahidulIslam2222